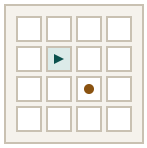
AP COMPUTER SCIENCE PRINCIPLES

Unit 1 — Karel

Performance Task · Design a Karel Solution



Name _____ Period _____ Due _____ **Time:** 3–4 class periods

PERFORMANCE TASK

Design a Karel Solution

This task mirrors the structure of the AP CSP Create Performance Task, but every requirement is built only from Unit 1 skills: sequencing, functions, top-down design, comments, abstraction, loops, and conditionals. You will build an original program, record it running, assemble a one-page reference, and write four short responses.

What you will turn in

1. **Program code** — your `.py` / CodeHS Karel file, meeting all six requirements below.
2. **Video** — a screen recording of the program running from start to finish (≤ 1 minute).
3. **Personalized Project Reference (PPR)** — one page with two labeled screenshots.
4. **Written Responses** — answers to WR 1–4, about 100–150 words each.

Part A — The Program

Pick **one** scenario (or propose your own for teacher approval). Each starts from a world you design and describe.

- **Variable-Height Skyline** — Karel builds a row of ball "towers"; how tall each tower is depends on a number your code controls.
- **Checkerboard Field** — Karel fills a rectangular region with balls in a checkerboard pattern.
- **Room Sweeper** — Karel is placed in a walled rectangular room and must visit every corner, picking up any ball it lands on.
- **Hurdle Runner** — Karel races east along Street 1, jumping every wall "hurdle" until it reaches the end, however far that is.

Your program must include all six of these:

1. **Instructions for the starting world (the "input").** In a comment at the top, state Karel's starting street and avenue, the direction it faces, the world size, where the walls are, and any balls already placed.
2. **A described result (the "output").** In a comment, state what the world and Karel look like when the program finishes.

3. **A student-developed procedure** that you define and name meaningfully, that has **at least one parameter that changes what it does** (for example, a number that controls how many times a loop repeats), *and* that contains **sequencing**, **selection** (an `if` or `if/else`), *and* **iteration** (a `for` or `while` loop).
4. **A call to that procedure**, passing an argument — for example `build_tower(4)`.
5. **Decomposition.** `main()` reads like an outline: a short list of calls to well-named functions, not one long stream of `move()` and `put_ball()`.
6. **Comments.** Each function has a comment saying what it does, and at least one function has a *pre-condition / post-condition* comment (where Karel is before, where Karel is after).

> **ABOUT PARAMETERS**
>
> Passing a value into a function (like `build_tower(4)`) goes one step past lessons 1.2–1.14, so your teacher will demo the pattern. The idea is small: a `for` loop already repeats a fixed number of times — a parameter just lets the *caller* choose that number. Model:

```
def build_tower(height):
    # pre: Karel faces East at the base of a column
    for i in range(height):
        put_ball()
        if front_is_clear():
            turn_left()
            move()
            turn_right()
    # post: a vertical stack of <height> balls is placed
```

Part B — The Video

- Screen-record CodeHS running your program from beginning to end.
- Show the world **before** you press Run and the finished world **after** it stops.
- No narration needed. Keep it under 1 minute; export as `.mp4` or `.gif`.

Part C — Personalized Project Reference (PPR)

One page (PDF or doc) with two clearly labeled screenshots taken from your own program:

1. **PPR 1:** your student-developed procedure — the whole definition, including the parameter in the `def` line.
2. **PPR 2:** the line where you *call* that procedure with an argument.

Short labels are fine; do not write full paragraphs on this page.

Part D — Written Responses

Answer in complete sentences, roughly 100–150 words each.

Written Response 1 — Program Purpose and Development

a. Describe the overall purpose of your program: what task does it make Karel complete?

b. Describe the starting world your program expects (the input) and the world it produces when it finishes (the output).

c. Describe one difficulty or design decision you ran into while building the program and how you worked through it.

Written Response 2 — The Procedure

a. Copy your student-developed procedure here.

b. Explain in detail how it works, step by step. Your explanation must clearly point out where it uses **sequencing**, **selection**, and **iteration**.

c. Explain how the **parameter** changes the procedure's behavior. Give two different argument values and describe how Karel acts differently for each.

Written Response 3 — The Algorithm

a. Inside your procedure, find a segment that is an algorithm made of a **loop that contains a conditional** (`if` or `if/else`). Copy just that segment.

b. Explain how the segment works and what it accomplishes for the program as a whole.

c. Explain why this segment could *not* be rewritten using sequencing alone — no loop and no conditional.

Written Response 4 — Testing

a. Describe **two different tests** of your program: two different starting worlds, or two different arguments passed to your procedure.

b. For each test, state the condition it checks and the exact result you expect Karel to produce.

c. Explain how these two tests together give you confidence that the program works correctly.

Scoring Rubric — 6 Points

Each row is worth 1 point. Rows mirror the Create Performance Task, adapted to Unit 1.

Pt	Row	Earn the point when…
1	Program & Purpose	The video shows the program running start to finish, and WR1 identifies the program's purpose and describes its input and output behavior.
2	World State	The program clearly changes Karel's world (position and/or balls), and WR1 accurately describes both the starting world and the ending world.
3	Procedure	The PPR and WR2 show a student-developed procedure *with a parameter that affects what it does*, containing sequencing, selection, *and* iteration.
4	Procedure Explanation	WR2 explains how the procedure works in detail *and* correctly explains the parameter's effect using two different argument values.
5	Algorithm	WR3 copies a loop-plus-conditional segment, explains how it works, and explains why sequencing alone will not do the job.
6	Testing	WR4 describes two distinct tests, the condition each one checks, the expected result, and how together they show the program is correct.

ACADEMIC INTEGRITY

The program design and all written responses must be your own work. You may ask a partner or your teacher for help finding a bug, but the structure of the program and the words in your responses are yours. If you use any code you did not write yourself, mark it with a comment that says where it came from.